AI RESUME ANALYZER

(CareerAI)

AI-Powered Resume Analysis, Resume Builder, Mock Interview Preparation and Job Application Management System

Project Documentation

Submitted by

NAME	REG NO	ROLL NO
GUNDAPANTHULA VENKATA SAI KRISHNA KOWSIK	12310647	75
BEHARA PRUDHVI	12312256	27
A. RAGHAVENDRA	12316617	43
K ADITYA SANKAR	12311083	23
JOGIPARTHI SANJAY	12309850	16

Table of Contents

List of Figures

List of Tables

Chapter 1 — Introduction

1.1 Background

Job seeking is a document-heavy and repetitive process. A candidate typically needs a well-formatted resume, an understanding of how that resume compares against a target job description, and preparation for the interview that may follow. Traditionally each of these steps is handled separately and manually: resumes are built in generic word processors, resume quality is judged subjectively by the candidate or a friend, and interview preparation is done through generic question banks that are not tailored to the candidate's own background or to the specific job being applied for.

AI Resume Analyzer (branded within the application as CareerAI) is a web application built to bring these separate activities — resume creation, resume analysis, mock interview preparation, job matching, and application tracking — into a single system, using Large Language Model (LLM) based AI assistance where it adds genuine value: generating resume content, generating interview questions, and comparing a resume against a job description.

1.2 Problem Statement

Job seekers, particularly students and early-career professionals, commonly face the following difficulties:

- Creating a professionally formatted resume without design or word-processing expertise.
- Knowing whether a resume is well-written, complete, and appropriately structured before submitting it.
- Understanding how a resume is likely to be read by Applicant Tracking Systems (ATS) and by human recruiters.
- Preparing for interviews in a way that is specific to their own resume and to the job description of the role being applied for, rather than relying on generic question lists.
- Judging how well a resume actually matches a particular job description before applying.
- Keeping track of which jobs have been applied to and the status of each application.

AI Resume Analyzer addresses these problems by combining a resume upload and parsing pipeline, AI-assisted analysis and content generation, an interview question generator that uses both the resume and the job description, a job-matching feature, and an application tracking module, all accessible from a single dashboard.

1.3 Objectives

Primary Objective

To design and develop a web-based system that helps job seekers create, analyze, and improve resumes, prepare for interviews, and evaluate how well their resume matches a given job description, using AI/LLM assistance where relevant.

Secondary Objectives

- Allow users to upload existing resumes in PDF or DOCX format and have the content extracted automatically.
- Provide AI-based analysis and feedback on an uploaded resume, including ATS-oriented observations.
- Provide both an AI-assisted resume builder and a manual resume builder, with selectable templates, theme colors, and fonts.
- Generate interview questions (with suggested answer guidance) based on a candidate's resume and a target job description, for a user-specified number of questions.
- Allow a user to compare a resume against a job description for a match assessment.
- Allow users to track jobs they have applied to, including company, role, status, and date.
- Provide a dashboard summarizing the user's resumes, applications, and interview preparation activity.
- Deploy the finished system so it is reachable as a live, publicly accessible web application.

1.4 Scope

The scope of the project, as implemented, covers resume upload and parsing (PDF/DOCX), AI-based resume analysis, AI-assisted and manual resume building with template, theme, and font selection, AI-generated interview questions driven by resume and job-description context, AI-based job matching between a resume and a job description, job application tracking, and a summary dashboard. The backend is built using Python and FastAPI, with SQLite as the database (accessed through SQLAlchemy) and the Groq API used as the LLM access layer. The frontend is a browser-based single-page client deployed separately from the backend and hosted at https://carrier-ai-frontend.onrender.com. Exact frontend implementation details that could not be independently confirmed from the deployed build are marked as "Not specified" throughout this report rather than assumed.

1.5 Motivation

AI language models are well suited to tasks that involve reading unstructured text — such as a resume — and producing structured, context-aware output, such as feedback, generated resume content, or interview questions. Building AI Resume Analyzer provided an opportunity to apply this capability to a genuinely useful, everyday problem — job search preparation — while also gaining practical, end-to-end experience in backend API development, database design, document parsing, prompt engineering, and integrating a third-party LLM API into a real, deployed application.

Note: This report documents AI Resume Analyzer based on the feature description, technology stack, and live deployment provided for this project. Items that would normally be confirmed by inspecting source code directly (exact API endpoints, database schema, and some frontend implementation details) are explicitly marked "Not specified" rather than assumed.

Chapter 2 — Existing System and Proposed System

2.1 Existing System

Conventionally, a job seeker builds a resume manually using a general-purpose word processor such as Microsoft Word or Google Docs, often starting from a downloaded template. Resume quality is assessed informally, either by the candidate themselves or by asking friends and mentors for feedback. Interview preparation is typically done by browsing generic, publicly available question lists that are not tailored to the candidate's resume or to the specific job. Comparing a resume against a job description, when done at all, is usually a manual, visual comparison. Tracking job applications is commonly done in an ad-hoc way, such as a spreadsheet, a notes app, or not at all.

2.2 Limitations of the Existing (Manual) Approach

- No automated or structured feedback on resume content or completeness.
- No AI-assisted generation of resume content — the user must write everything themselves.
- Interview preparation is generic and not personalized to the candidate's actual resume or the job description.
- No systematic way to gauge resume-to-job-description fit before applying.
- No centralized tracking of job applications and their outcomes.
- Resume formatting and design work is repeated from scratch for every new version of the resume.
- Feedback quality depends entirely on the availability and expertise of whoever the candidate asks.

2.3 Proposed System — AI Resume Analyzer

AI Resume Analyzer proposes a single web application that brings resume upload, AI-based resume analysis, AI-assisted and manual resume building (with templates, theme, and font selection), AI-generated interview preparation, AI-based job matching, and job application tracking together, backed by a FastAPI backend, a SQLite database, and the Groq API as the LLM access layer, with a browser-based frontend deployed independently as a hosted web client.

2.4 Advantages of the Proposed System

- AI-assisted resume content generation reduces the effort of writing a resume from scratch.
- AI-based resume analysis gives the user structured feedback rather than relying only on self-assessment.
- Interview questions are generated using both the resume and the job description, making preparation more relevant to the specific application being made.
- Job matching gives the user a way to assess fit before applying, rather than after being rejected.
- A single dashboard consolidates resumes, applications, and preparation activity in one place.

- The system is accessible from any modern browser as a hosted web application, with no local installation required by the end user.

2.5 Existing System vs Proposed System — Comparison

Area	Traditional / Manual Approach	AI Resume Analyzer
Resume creation	Manual, in a general word processor	AI-assisted builder or manual builder with templates
Resume analysis	Informal / self-assessed	AI-based analysis on extracted resume text
AI assistance	None	LLM-based content generation and analysis via Groq API
Interview preparation	Generic question lists	Questions generated from the user's resume and job description
Job matching	Manual visual comparison, if done at all	AI-based comparison of resume and job description
Job tracking	Ad-hoc (spreadsheet or none)	Dedicated application tracking module
Customization	Limited to word processor formatting	Template, theme color, and font selection
Personalization	None	Content and questions generated per user resume / job description
Accessibility	Local files on one device	Hosted web application, reachable from any browser

Chapter 3 — Requirements Analysis

3.1 Functional Requirements

The system shall support the following functions:

1. The user can upload a resume in PDF or DOCX format.
2. The system can extract text content from an uploaded resume.
3. The user can request AI-based analysis of an uploaded resume.
4. The user can build a resume using the AI Resume Builder by selecting a template, theme color, and font, entering a resume title, and providing instructions to the AI.
5. The user can build a resume manually using the Manual Resume Builder, selecting a template, theme color, and font.
6. The system can generate resume content using AI based on user-provided instructions.
7. The user can preview the resume before saving.
8. The user can save the generated or manually built resume.
9. The user can print or download the resume.
10. The user can request interview questions by providing a resume, a job description, and the desired number of questions.
11. The system generates the requested number of interview questions, each with suggested approach or answer guidance.
12. The user can request a job-matching analysis by providing a resume and a job description.
13. The user can track jobs they have applied to, including company, title, status, and date.
14. The user can view a dashboard summarizing resumes, applications, and interview preparation activity.
15. The user can access the application over the internet through the deployed frontend URL without installing any software.

3.2 Non-Functional Requirements

Requirement	Description	Status
Performance	The system should respond to user actions within a reasonable time; AI-dependent operations (analysis, generation, matching) are bound by third-party API response time.	Dependent on Groq API latency — not formally benchmarked
Usability	The interface should allow a user with no technical background to upload, build, and manage resumes.	Design goal
Reliability	The system should handle invalid input (unsupported files, missing fields) gracefully.	Recommended improvement — see Chapter 9

Requirement	Description	Status
Maintainability	Code should be organized into routes, services, schemas, and database layers for ease of maintenance.	Reflected in the described backend folder structure
Security	API keys and sensitive configuration should not be exposed to the client.	Implemented via .env / python-dotenv on the backend; further hardening is a future goal
Scalability	The system should be able to handle a growing number of users and resumes.	Future goal — SQLite is not designed for large-scale concurrent production use
Availability	The system should be available whenever the hosted backend and frontend services are running.	Applies to the current Render-hosted deployment; no formal uptime guarantee
Portability	The frontend should run in any modern desktop or mobile browser without additional plugins.	Design goal for a browser-based single-page client

3.3 Hardware Requirements

- End user: any device with a modern web browser and an internet connection (desktop, laptop, tablet, or smartphone).
- Development: a computer capable of running Python 3, Node-based tooling for the frontend, and a modern web browser.
- Server: cloud compute sufficient to run a Python/Uvicorn process and serve a static or server-rendered frontend build (provided by the Render hosting platform in the current deployment).
- Internet connectivity is required at all times for the Groq API calls used by the AI features.
- Exact minimum hardware specification for self-hosting: Not specified.

3.4 Software Requirements

- Python 3.x runtime for the backend.
- FastAPI framework and Uvicorn ASGI server.
- SQLite database engine.
- SQLAlchemy ORM.
- python-dotenv for environment variable management.
- Groq Python client, and a valid Groq API key.
- A PDF parsing library and a DOCX parsing library (exact libraries used: Not specified).

- A frontend build toolchain capable of producing a deployable single-page application (exact framework: Not specified — see Chapter 4.9).
- Render (or an equivalent PaaS) for hosting both the frontend and backend services.

Note: Exact version numbers of the above libraries, and the specific PDF/DOCX parsing libraries used, were not confirmed from source inspection and are therefore not stated as fact anywhere in this report.

3.5 User Requirements

Two broad classes of user were considered while defining requirements. The first is a job seeker who wants to build a new resume from scratch with AI assistance and has limited design experience. The second is a job seeker who already has an existing resume, wants an objective analysis of it, wants to check it against a specific job description, and wants to prepare for the resulting interview. Both classes of user are expected to interact with the system without any technical knowledge of AI, prompt engineering, or web development, meaning the interface must abstract away all of the underlying LLM and API mechanics described in Chapter 7.

3.6 Assumptions and Constraints

- The system assumes uploaded resumes are primarily text-based PDF/DOCX files rather than scanned images, since OCR is not part of the described parsing pipeline.
- The system assumes continuous internet connectivity and Groq API availability for all AI-dependent features to function.
- The system is constrained by SQLite's single-writer design, which is acceptable for the current scale but is a known limitation for concurrent production use (see Chapter 10.3).
- The system currently has no confirmed authentication layer, which constrains how resume and application data can be safely scoped to individual users (see Chapter 11).

Chapter 4 — Technology Stack

This chapter describes each technology used in the project and explains, in simple terms, what role it plays. Technologies confirmed for this project are described in detail; anything not independently confirmed is marked accordingly rather than assumed.

4.1 Python

Python is a general-purpose, high-level programming language known for readable syntax and a large ecosystem of libraries. It is used here to implement the entire backend of AI Resume Analyzer: the web server logic, file handling, database access, and the code that communicates with the Groq API.

4.2 FastAPI

FastAPI is a modern Python web framework used to build APIs. It allows routes (URL endpoints) to be defined as ordinary Python functions, automatically validates request and response data using type hints and Pydantic-style schemas, and automatically generates interactive API documentation. In AI Resume Analyzer, FastAPI is the framework that exposes the backend's functionality — upload, analyze, improve, interview, and job tracking — to the frontend over HTTP.

4.3 Uvicorn

Uvicorn is an ASGI (Asynchronous Server Gateway Interface) server used to actually run a FastAPI application. It is the process that listens for incoming HTTP requests and passes them to the FastAPI application code. FastAPI defines what happens when a request arrives; Uvicorn is what keeps the application running and reachable, including in the hosted deployment.

4.4 SQLite

SQLite is a lightweight, file-based relational database engine that does not require a separate database server process — the entire database lives in a single file. It is well suited to development, prototyping, and small-to-moderate scale applications because it requires no separate installation or server management, while still supporting standard SQL operations.

4.5 SQLAlchemy

SQLAlchemy is a Python Object-Relational Mapper (ORM). An ORM allows database tables to be represented as Python classes and rows as Python objects, so the application code can read and write data using normal Python syntax instead of writing raw SQL for every operation. This also makes the code more portable if the underlying database were ever changed.

4.6 Groq API and the Role of the LLM

Groq provides an API and inference platform through which a Large Language Model (LLM) can be accessed programmatically. Groq itself is not the AI model; it is the service that runs and exposes the model over an API, and is used here for its response-speed characteristics. The backend's AI service loads a GROQ_API_KEY from environment variables, creates a client using the Groq Python client library, and sends prompts (system and user messages) to the LLM through this client. The specific LLM model name and version used through Groq: Not specified.

An LLM is a type of AI model trained on large amounts of text that can read an input prompt and generate relevant text as output — for example, reading a resume and job description and generating interview questions, or reading resume text and generating structured feedback. AI Resume Analyzer relies on the LLM's ability to read unstructured resume and job-description text and produce structured, relevant output.

4.7 PDF and DOCX Processing

The backend includes parsing logic for both PDF and DOCX resume files, used to extract plain text content from an uploaded resume so it can be used by the AI-based features. The exact third-party parsing libraries used inside these modules: Not specified.

4.8 Environment Configuration

python-dotenv is used to load configuration values, such as the Groq API key, from a .env file into environment variables at runtime, keeping secrets out of the source code itself.

4.9 Frontend Technology

The deployed frontend is served as a hosted single-page web client at https://carrier-ai-frontend.onrender.com, branded within the application as "CareerAI — Smart Resume & Job Management." The site is deployed as a separately hosted service from the backend, consistent with a decoupled frontend/backend architecture where the browser client communicates with the FastAPI backend over HTTP/JSON. The exact frontend framework or library used (for example, a specific JavaScript framework, plain HTML/CSS/JS, or another approach) could not be independently confirmed from the deployed build and is not assumed in this report; it should be confirmed and filled in directly from the frontend project's own source or package.json before final submission.

4.10 Deployment Platform

The application is hosted on Render, a cloud platform-as-a-service. The naming of the deployed frontend service ("carrier-ai-frontend") is consistent with a companion backend service being deployed separately under a similar naming convention, which matches the decoupled architecture described in Chapter 5. Render was chosen, or is at least being used, for its ability to host both a Python/Uvicorn backend process and a static or server-rendered frontend build without requiring manual server administration.

4.11 Technology Stack Summary

Layer	Technology	Purpose
Backend language	Python	Implements the application logic
Web framework	FastAPI	Defines and exposes API routes
Application server	Uvicorn	Runs the FastAPI application
Database	SQLite	Stores application data
ORM	SQLAlchemy	Maps Python objects to database tables
AI / LLM access	Groq API (Groq Python client)	Sends prompts to an LLM and receives generated text
Configuration	python-dotenv (.env)	Loads secrets and configuration at runtime
Document processing	PDF parser, DOCX parser (libraries not specified)	Extracts text from uploaded resumes
Frontend	Hosted single-page web client (framework not specified)	User interface, served at carrier-ai-frontend.onrender.com
Hosting / deployment	Render (PaaS)	Hosts the frontend and backend services

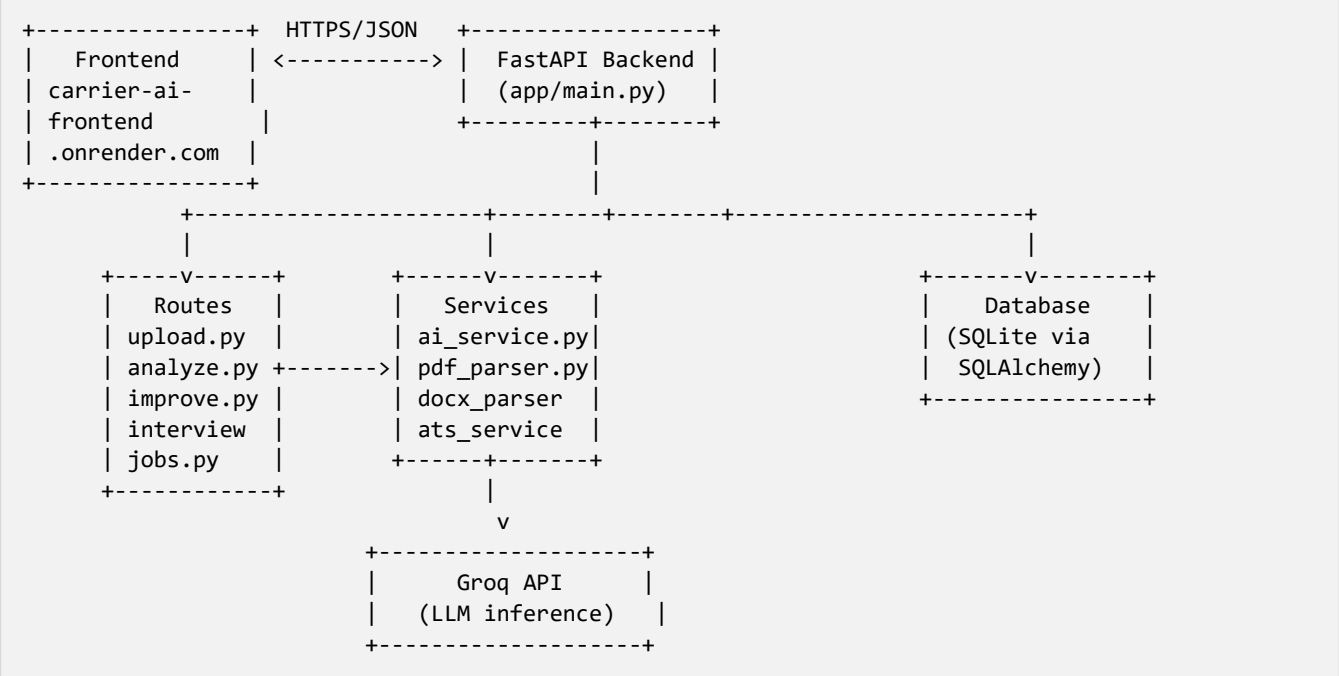
Note: No frontend framework, specific parsing library, or LLM model version is assumed beyond what could be confirmed from the deployed application and its metadata. These should be confirmed and filled in directly from the project's source files (for example, requirements.txt and package.json) before final submission.

Chapter 5 — System Design

This chapter presents the major design views of AI Resume Analyzer. Each diagram is described in terms of its purpose and components, followed by a simple text sketch that can be recreated in a diagramming tool such as draw.io, Lucidchart, or Mermaid.

5.1 System Architecture Diagram

Purpose: shows the high-level layers of the system and how they connect — the browser-based frontend client, the FastAPI backend, the service layer, the AI provider (Groq API), and the database.

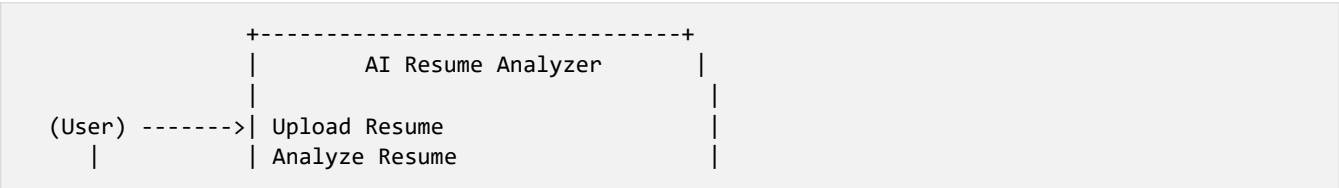


5.2 Data Flow Diagram (Level 0 / Context)

Purpose: shows how data moves between the user, the system, and external services at a high level.

```
User ----(resume file / job description / instructions)----> Frontend
Frontend ----(HTTPS/JSON request)----> AI Resume Analyzer Backend
Backend ----(prompt)----> Groq API (LLM)
Groq API (LLM) ----(generated text / JSON)----> Backend
Backend ----(stored records)----> Database (SQLite)
Backend ----(analysis / resume / questions / match result)----> Frontend ----> User
```

5.3 Use Case Diagram



```
     |---------->| Build Resume (AI)               |
     |---------->| Build Resume (Manual)           |
     |---------->| Prepare for Interview           |
     |---------->| Match Resume to Job              |
     |---------->| Track Job Applications          |
     +---------->| View Dashboard                  |
                 +---------------------------------+
```

5.4 Component Diagram

```
[Routes Layer] --uses--> [Services Layer] --uses--> [Database Layer]
  upload.py                pdf_parser.py               models.py
  analyze.py               docx_parser.py               database.py
  improve.py               ai_service.py --calls--> [Groq API]
  interview.py             ats_service.py
  jobs.py                  improvement_service.py

[Schemas] <-- request/response validation used by -- [Routes Layer]
[Prompts] <-- prompt templates used by -- [ai_service.py]
[Utils]   <-- shared helper functions used across -- [Services / Routes]
```

5.5 Database / Entity-Relationship Diagram

Purpose: shows the entities managed by the application and their relationships, as far as can be inferred from the described features. Exact table and column names could not be confirmed without inspecting the database model definitions directly — see Appendix A for details and limitations.

```
+----------------+          +------------------------+
|    Resume      |          |   InterviewSession     |
|----------------|          |------------------------|
| resume_id (PK) |<-------->| resume_id (FK)         |
| ...            |          | job_description        |
+----------------+          | num_questions           |
                            +------------------------+
+----------------+          +------------------------+
| JobApplication |          |   JobMatchResult       |
|----------------|          |------------------------|
| resume_id (FK) |          | resume_id (FK)          |
| company, title,|          | job_description         |
| status, date   |          | match analysis          |
+----------------+          +------------------------+
```

5.6 Resume Processing Flowchart

```
User uploads resume
        |
        v
FastAPI receives file (upload.py)
        |
        v
File validation (type / size)
        |
```

```
        v
Resume stored (uploads/ + DB record)
        |
        v
PDF/DOCX parser extracts text
        |
        v
Extracted resume content available
        |
        v
Used by: Analysis / Improvement / Interview / Matching
        |
        v
Structured response returned to frontend
```

5.7 AI Resume Generation Flowchart

```
User enters title, instructions, template, theme, font
        |
        v
Frontend sends request to FastAPI backend
        |
        v
Route (builder / improve.py) receives request
        |
        v
ai_service.py builds prompt and calls Groq API
        |
        v
LLM returns generated resume content (structured)
        |
        v
Content applied to selected template
        |
        v
Live preview rendered in the browser
        |
        v
User saves / prints / downloads the resume
```

5.8 Interview Preparation Flowchart

```
User provides: resume + job description + number of questions
        |
        v
FastAPI (interview.py) receives request
        |
        v
ai_service.py builds prompt (resume + JD + N context)
        |
        v
Groq API / LLM generates questions
        |
        v
```

```
Response validated: does question count == N?
     |                |
    yes               no
     |                |
     v                v
Return structured   Retry / adjust prompt
questions + guidance (recommended improvement)
        |
        v
Frontend displays interview questions
```

5.9 Job Matching Flowchart

```
User provides: resume + job description
        |
        v
Resume text retrieved (already extracted / re-parsed)
        |
        v
ai_service.py sends resume + JD to LLM for comparison
        |
        v
LLM compares skills / experience / requirements
        |
        v
Match result (analysis, and score if implemented)
        |
        v
Frontend displays match result
```

5.10 Deployment Diagram

Purpose: shows how the two deployed services (frontend and backend) relate to each other and to external dependencies in the live environment.

```
  [Browser]
      |  HTTPS
      v
  [Render: carrier-ai-frontend]  --- HTTPS/JSON --->  [Render: backend service]
                                                              |
                                                              v
                                                    [SQLite file: resume.db]
                                                              |
                                                              v
                                                       [Groq API (LLM)]
```

Chapter 6 — Implementation

This chapter describes how each feature is implemented, structured around the backend architecture. The backend folder structure, as described for this project, is shown below.

```
backend/
|
+-- app/
|   +-- main.py
|   |
|   +-- routes/
|   |   +-- upload.py
|   |   +-- analyze.py
|   |   +-- improve.py
|   |   +-- interview.py
|   |   +-- jobs.py
|   |
|   +-- services/
|   |   +-- pdf_parser.py
|   |   +-- docx_parser.py
|   |   +-- ai_service.py
|   |   +-- ats_service.py
|   |   +-- improvement_service.py
|   |
|   +-- database/
|   |   +-- models.py
|   |   +-- database.py
|   |
|   +-- schemas/
|   +-- utils/
|   +-- prompts/
|
+-- uploads/
+-- generated/
+-- resume.db
+-- requirements.txt
+-- .env
```

6.1 Purpose of Each Backend Module

Folder / File	Purpose
main.py	Entry point of the FastAPI application; registers routes and starts the app.
routes/	Defines the API endpoints (HTTP routes) that the frontend calls.
services/	Contains the business logic — parsing, AI calls, analysis — kept separate from the routing layer.
database/	Defines the SQLAlchemy models (tables) and the database connection/session setup.
schemas/	Defines request/response data shapes (Pydantic models) used for validation.
utils/	Shared helper functions used across the application.
prompts/	Prompt templates used when constructing requests to the Groq API.

Folder / File	Purpose
uploads/	Stores uploaded resume files.
generated/	Stores AI-generated or rendered resume output.
resume.db	The SQLite database file.
requirements.txt	Lists Python package dependencies.
.env	Stores environment variables such as the Groq API key (not committed to version control).

6.2 Resume Upload

The user uploads an existing resume in PDF or DOCX format through the upload interface. Resume upload is the entry point for all AI-assisted features that require the candidate's actual background — without extracted resume content, the system has nothing specific to analyze, build from, or use for interview or matching context. On the backend, the route in upload.py receives the file, and the appropriate parser is used depending on the file type to extract plain text. The uploaded file is stored and associated with a resume identifier so it can be referenced later by other features without re-uploading.

6.3 Resume Parsing (Text Extraction)

PDF and DOCX files store text in structured, format-specific ways rather than as plain readable text. Parsing extracts the readable text content from these formats so it can be passed to the AI service as a plain string. The specific parsing libraries used inside the PDF and DOCX parser modules: Not specified.

6.4 Resume Analysis

Once resume text has been extracted, the analyze route allows the user to request an AI-based analysis of the resume. The extracted text is passed to the AI service, which sends it to the LLM via Groq with a prompt requesting feedback. Aspects of analysis such as skills, experience, education, and projects can be covered because they are all present in typical resume text; the exact scope of the analysis prompt actually implemented, including whether ATS-specific scoring is included, is handled by the ATS and improvement service modules — exact analysis criteria implemented: Not specified beyond what these module names indicate.

6.5 AI Resume Builder

The AI Resume Builder lets the user select a resume template, a theme color, and a font; enter a resume title; and provide instructions describing the resume content they want, such as career background or target role. This is sent to the backend, which uses the AI service to construct a prompt from the user's instructions and requests generated resume content from the LLM. The generated content is applied to the selected template for a live preview, which the user can review before saving, printing, or downloading the resume.

Templates

The application includes multiple resume templates, including Classic Sidebar, Dark Banner, and Minimal. Template selection determines which layout is used to render the resume; the selected template is reflected in the live preview so the user sees the final appearance before saving.

6.6 Manual Resume Builder

The Manual Resume Builder allows the user to build a resume by entering content themselves rather than having it AI-generated, while still selecting a template, theme color, and font. The resume preview updates according to the selected configuration, using the same template rendering used by the AI builder. Providing both an AI Resume Builder and a Manual Resume Builder accommodates two different user needs: users who want AI assistance to draft content quickly, and users who prefer full control over their own wording but still want template-based formatting and a professional layout.

6.7 Template System

The template system separates resume content (data) from resume presentation (layout and styling). The same underlying resume data — whether AI-generated or manually entered — can be rendered using any of the available templates, with the selected theme color and font applied. This is what allows a live preview to update immediately when the user changes the template, theme, or font.

6.8 Interview Preparation

The user provides a resume, a job description, and the number of interview questions they want generated. The interview route passes this information to the AI service, which constructs a prompt combining resume context and job description context, and requests the LLM to generate that exact number of interview questions, each with suggested approach or answer guidance. Including the job description in the prompt, rather than generating generic questions, allows the questions to reflect the specific role's requirements, not just the candidate's general background. Ensuring the system returns exactly the number of questions requested is discussed further in Chapter 9 and Chapter 10, since LLMs do not always follow numeric instructions exactly, so response validation — and, if needed, a retry — is relevant here.

6.9 Job Matching

The user provides a resume and a job description. The system uses the AI service to compare the resume content against the job description, considering skills, experience, and other requirements mentioned in the job description, and returns an analysis of how well they match. Whether a specific numeric match score is generated, as opposed to a purely qualitative analysis, depends on the actual prompt and response design implemented; this report does not assume a specific scoring algorithm exists unless confirmed.

6.10 Job Application Tracking

The job tracking module lets users record and track jobs they have applied to. Tracking applications in one place is useful because job searches typically involve applying to many roles over time, and it becomes difficult to remember which resume version, company, role, and status apply to each application without a

dedicated record. Fields such as company, job title, status, and date are consistent with the feature description provided; exact fields implemented in the database should be confirmed from the model definitions.

6.11 Dashboard

The dashboard provides a single overview screen summarizing the user's activity: resumes uploaded, resumes built, jobs applied to, interviews prepared, recent resumes, recent job applications, and quick actions to jump into a specific feature. Its role in the overall application is to reduce the need to navigate through each module individually just to see a summary of progress, and to serve as the natural landing page after opening the application.

Chapter 7 — AI Integration

7.1 LLM Integration

AI Resume Analyzer integrates AI functionality through the Groq API, accessed via the Groq Python client inside the AI service module. The API key is loaded from environment variables using python-dotenv, and a Groq client instance is created using this key. Every AI-dependent feature — resume analysis, AI resume content generation, interview question generation, and job matching — routes through this shared AI service rather than each feature implementing its own API calls, which keeps the prompt-building and API-handling logic centralized.

7.2 Prompt Engineering

Prompt engineering refers to designing the text sent to an LLM so that its response is accurate, relevant, and in the expected format. In AI Resume Analyzer, prompts are built from two parts: a system message that sets the LLM's role and instructions — for example, instructing it to act as a resume and interview assistant and to respond in a specific structure — and a user message that contains the actual context for the request, such as extracted resume text, a job description, or builder instructions.

Context Injection

- Resume context: extracted resume text is injected into the prompt for analysis, AI building, interview preparation, and job matching.
- Job description context: injected for interview question generation and job matching, so output is specific to the target role rather than generic.
- User instructions: injected for the AI Resume Builder, so generated content reflects what the user actually asked for.
- Question count: injected for interview preparation, instructing the LLM to generate a specific number of questions.

Keeping prompt templates as separate, reusable artifacts rather than hard-coding them inline throughout the service logic makes them easier to review and adjust independently of the application code.

7.3 Structured Output (JSON)

Where appropriate, the AI service requests structured JSON responses from the LLM rather than free-form text. This matters because the backend needs to programmatically use the LLM's output — for example, to populate specific resume template fields, to count how many interview questions were generated, or to display a job match analysis in a specific UI layout. Free-form text would need to be parsed unreliably after the fact; asking the LLM to directly return JSON in an agreed structure lets the FastAPI backend parse the response directly into Python objects and pass it on to the frontend.

7.4 Resume Analysis (AI Use)

For resume analysis, extracted resume text is sent to the LLM with a prompt asking for feedback relevant to resume quality, such as structure, completeness, and clarity. The LLM's response is returned to the frontend as the analysis result.

7.5 Resume Generation (AI Use)

For AI-assisted resume building, the user's instructions and any resume context, if provided, are sent to the LLM with a prompt requesting resume content organized in a way that can populate the selected template's fields, such as summary, experience, education, and skills.

7.6 Interview Question Generation (AI Use)

For interview preparation, the resume text, job description, and requested question count are combined into a single prompt. The LLM is asked to generate that number of questions, each paired with suggested approach or answer guidance, ideally returned as a structured list so the frontend can render them individually.

7.7 Job Matching (AI Use)

For job matching, the resume text and job description are sent to the LLM with a prompt asking it to compare the two, identifying overlapping and missing skills, relevant experience, and other job requirements, and to return an analysis of the fit between the resume and the role.

7.8 Why an API-Based LLM Rather Than a Self-Hosted Model

Using an API-based LLM via Groq instead of hosting a model directly avoids the need to provision GPU infrastructure, manage model weights, or handle model serving, which is impractical for a student or academic project. It also allows the project to benefit from the capability of a larger, well-trained model than could realistically be trained or hosted independently.

7.9 Prompt Design Principles Applied

- Role clarity: the system message establishes the LLM's role (for example, resume assistant or interview coach) before any task-specific instruction is given.
- Explicit output format: prompts specify the exact structure expected in the response, reducing ambiguity in downstream parsing.
- Context grounding: every generated output is grounded in user-supplied context (resume text, job description) rather than being produced from the prompt alone, which keeps output personalized.
- Constraint statement: numeric or structural constraints, such as the requested interview question count, are stated explicitly and are the basis for post-response validation.

Chapter 8 — User Interface and User Experience

This chapter documents the application from the end user's point of view, screen by screen, based on the feature set exposed through the deployed frontend at https://carrier-ai-frontend.onrender.com, branded as CareerAI — Smart Resume & Job Management. Exact visual layout, colors, and component-level detail were not independently captured as screenshots for this report and should be inserted from the live application before final submission.

8.1 Overall Navigation Model

The application is organized as a single-page client with a persistent navigation structure leading to each major feature area: Dashboard, Resume Analysis, Resume Builder (Manual and AI), Interview Preparation, Job Matching, and Applied Jobs. This flat navigation model keeps every major feature reachable within one or two clicks from any screen, which is appropriate for a task-oriented tool where users move between related activities, such as analyzing a resume and then generating interview questions for the same resume.

8.2 Screen Inventory

Screen	What the User Can Do
Landing / Home	View the product's value proposition — build, analyze, and optimize a resume, practice mock interviews, and manage job applications — and enter the application.
Dashboard	View a summary of resumes uploaded or built, jobs applied to, interviews prepared, recent resumes, recent applications, and access quick actions.
Resume Analysis	Upload or select a resume and request AI-based analysis and feedback.
Resume Builder (Manual)	Manually enter resume content, choose a template, theme color, and font, and preview the result.
AI Resume Builder	Provide a title and instructions to the AI, choose a template, theme, and font, generate content, preview it, save it, and print or download it.
Interview Preparation	Provide a resume, a job description, and the number of questions; view generated questions and answer guidance.
Job Matching	Provide a resume and a job description; view the AI-generated match analysis.
Applied Jobs	View and manage a list of jobs the user has applied to, including status tracking.

8.3 Resume Upload and Analysis Flow

From the user's perspective, this flow begins with selecting a PDF or DOCX file from their device, after which the frontend uploads it to the backend and displays a progress or loading state while text extraction and, if requested, AI analysis take place. The resulting feedback is displayed in a structured format so the user can act on specific points rather than reading an undifferentiated block of text.

8.4 Resume Builder Flow (AI and Manual)

For the AI Resume Builder, the user first selects a template, theme color, and font, then enters a title and free-text instructions describing what they want the resume to contain. Submitting this triggers a backend request that returns generated content, which is applied live to the chosen template so the user can see the final appearance immediately. For the Manual Resume Builder, the same template, theme, and font controls are available, but the user types the resume content directly rather than generating it. In both flows, the user can preview, save, print, or download the finished resume.

8.5 Interview Preparation Flow

The user supplies a resume (previously uploaded or built within the system), pastes or types a target job description, and specifies how many interview questions they want. After submission, the interface displays the generated questions, each accompanied by suggested guidance on how to approach an answer, allowing the user to work through them as a self-paced practice session.

8.6 Job Matching Flow

The user supplies a resume and a job description and requests a match analysis. The result is presented as a structured comparison highlighting where the resume aligns with the role and where gaps exist, giving the user actionable information before deciding whether, or how, to apply.

8.7 Applied Jobs Flow

The user can add a job application record with details such as company, role, and status, and can review or update these records over time. This turns what is normally tracked in an external spreadsheet into a feature native to the same tool used for resume preparation.

8.8 Usability Considerations

- Loading and progress states are important throughout the application because every AI-dependent action depends on third-party API latency (see Chapter 3.2).
- Structured, scannable output (lists, sections, headings) is preferable to long free-form paragraphs for analysis, interview questions, and match results, since users are typically comparing multiple pieces of feedback at once.
- Live preview in the resume builders reduces the need for a separate save-and-check cycle, letting users see the effect of template, theme, and font changes immediately.
- Consistent navigation across all feature screens reduces the learning curve for users who move frequently between analysis, building, interview preparation, and job tracking.

8.9 Accessibility and Responsiveness

As a browser-based single-page application intended to be used on both desktop and mobile devices, the interface should support responsive layout behaviour so that forms, previews, and generated content

remain usable on smaller screens. Specific accessibility conformance (for example, WCAG level) was not confirmed for this report and is listed as a recommended verification item in Chapter 11.

Chapter 9 — Testing, Validation and Quality Assurance

9.0 Testing Strategy

As no automated test suite or test execution results were provided for this project, this section presents a recommended testing strategy — the test cases the project should be verified against — rather than reporting actual pass/fail results. This distinction is intentional, in line with the principle of not fabricating performance or test metrics that were not actually measured.

9.1 Test Cases — Resume Upload

Test Case	Input	Expected Result
Valid PDF upload	A well-formed .pdf resume file	File accepted, stored, and text extracted successfully
Valid DOCX upload	A well-formed .docx resume file	File accepted, stored, and text extracted successfully
Invalid file type	A file with an unsupported extension (.txt, .jpg)	Upload rejected with an appropriate error message
Missing file	Upload request sent with no file attached	Request rejected with a validation error
Oversized file	A file larger than the intended size limit	Upload rejected with a clear size-limit message

9.2 Test Cases — Resume Analysis

Test Case	Input	Expected Result
Valid resume analysis	A valid resume identifier for an uploaded resume	AI analysis returned successfully
Invalid resume ID	A resume identifier that does not exist	Error returned indicating the resume could not be found
Empty resume	A resume file with little or no extractable text	Graceful handling — a meaningful error or a low-confidence analysis, not a crash

9.3 Test Cases — AI Resume Builder

Test Case	Input	Expected Result
Valid instructions	Complete builder instructions, template, theme, font	Resume content generated and rendered in the selected template
Missing instructions	Empty or missing instructions field	Validation error, or a sensible default prompt behaviour
Template selection	Switching between Classic Sidebar, Dark Banner, Minimal	Preview updates to reflect the newly selected template
Theme selection	Changing theme color	Preview updates with the new theme color

Test Case	Input	Expected Result
Font selection	Changing font	Preview updates with the new font

9.4 Test Cases — Interview Preparation

Test Case	Input	Expected Result
Valid resume and job description	Valid resume ID and non-empty job description	Interview questions and guidance generated
Different question counts	Requesting, for example, 5 vs 10 questions	Exactly the requested number of questions returned
Missing job description	Empty job description field	Validation error returned
Invalid resume	A resume identifier that does not exist	Error returned indicating the resume could not be found
AI failure	Groq API unreachable or returns an error	Backend returns a handled error, not an unhandled crash

9.5 Test Cases — Job Matching

Test Case	Input	Expected Result
Valid resume and job description	Valid resume ID and non-empty job description	Match analysis returned
Invalid resume	A resume identifier that does not exist	Error returned indicating the resume could not be found
Missing job description	Empty job description field	Validation error returned
AI failure	Groq API unreachable or returns an error	Backend returns a handled error, not an unhandled crash

9.6 Test Cases — Database

Test Case	Input	Expected Result
Record creation	Valid data for a resume, application, or interview record	Record persisted and retrievable
Record retrieval	A valid record identifier	Correct record returned
Invalid ID	A non-existent record identifier	Handled error (for example, a 404 response), not an unhandled exception

9.7 Test Cases — Deployment / End-to-End

Test Case	Input	Expected Result
Frontend reachability	Navigate to https://carrier-ai-frontend.onrender.com	Application loads and the landing page renders correctly

Test Case	Input	Expected Result
Frontend-to-backend connectivity	Perform an action that requires a backend call (upload, analyze)	Request completes and the result is displayed without a network error
Cold-start latency	Access the application after a period of inactivity on a free-tier host	Application eventually loads; delay should be communicated to the user via a loading state

9.8 Validation Approach

Validation should occur at two levels: request-level validation using FastAPI and schema validation, to reject malformed requests before they reach business logic; and response validation for AI-generated content — for example, checking that the number of interview questions returned matches the number requested before returning the response to the frontend.

9.9 Error Handling Strategy

Error Scenario	Recommended Handling	Status
Invalid resume file	Return a clear error indicating the file could not be processed	Recommended
Unsupported file format	Reject before parsing, with a specific error message	Recommended
Missing resume	Return a 404-style error for the given identifier	Recommended
Invalid resume ID	Return a 404-style error rather than an unhandled exception	Recommended
Empty job description	Reject at validation stage with a clear message	Recommended
Missing API key	Fail fast at startup or on first AI call with a clear configuration error	Recommended
AI API errors (unreachable / rate-limited)	Catch the exception and return a handled error response, not a raw stack trace	Recommended
Invalid or malformed AI response	Validate the response before using it; return an error if parsing fails	Recommended
Database errors	Catch exceptions from the ORM layer and return a handled error	Recommended
Incorrect interview question count	Validate count against the request; retry or trim/pad with a clear note if mismatched	Recommended — see Chapter 10

Note: The table above lists recommended error-handling behaviour. Whether each of these is already implemented in the codebase was not confirmed and should be verified against the actual route and service code before this section is finalized.

Chapter 10 — Security Considerations

This chapter distinguishes between security measures that are implemented, as described for this project, and those that are recommended improvements but not confirmed as implemented.

10.1 Implemented

- API key protection: the Groq API key is kept in a .env file and loaded via python-dotenv, rather than being hard-coded into source files.
- Separation of frontend and backend as independently deployed services, which limits direct exposure of backend internals through the client bundle.

10.2 Recommended Improvements (Not Confirmed as Implemented)

- File type validation on upload beyond basic extension or content-type checks — for example, verifying actual file content matches the claimed type.
- File upload size limits and stricter upload security, such as scanning or sandboxed storage.
- Systematic input validation across all routes using schema validation for every field.
- Authentication and authorization — the project description does not confirm a login/authentication system (see Chapter 11, Future Enhancements).
- Database-level security hardening, such as encryption at rest, for the SQLite file.
- Safe handling and sanitization of AI-generated output before it is rendered in the frontend, to avoid issues such as unexpected formatting or injection when displaying LLM output.
- Rate limiting on AI-dependent endpoints to control API usage and cost.
- HTTPS enforcement and secure headers on both the frontend and backend hosted services.
- CORS configuration limited to the known frontend origin, rather than an open wildcard policy.

Note: No security features are described as implemented beyond .env-based API key storage and service-level separation. All other items above should be treated as recommended future work, not existing functionality, unless confirmed otherwise from source code.

10.3 Data Privacy Considerations

Resumes and job descriptions are personal and sometimes sensitive documents. Because the current system has no confirmed authentication layer, resume and application data appears to be identified by record or resume identifiers rather than being scoped to individual user accounts. This is a privacy-relevant limitation: without authentication, there is no confirmed mechanism preventing one user's resume identifier from being guessed or accessed by another party. Introducing authentication, as listed in Chapter 11, would also allow data access to be properly scoped per user.

10.4 Third-Party Data Exposure

Resume text and job descriptions are sent to the Groq API as part of every AI-dependent request. Users should be informed, at least in a privacy notice, that this content leaves the application's own infrastructure and is processed by a third-party LLM provider, consistent with standard practice for any application that integrates a third-party AI API.

Chapter 11 — Results and Discussion

11.1 Features Implemented

The following features are described as implemented in AI Resume Analyzer:

- Resume upload (PDF and DOCX) with text extraction.
- AI-based resume analysis.
- AI Resume Builder with template, theme color, and font selection, AI-generated content, live preview, save, and print/download.
- Manual Resume Builder with template, theme color, and font selection.
- Interview Preparation — AI-generated questions with suggested guidance, based on resume and job description, for a user-specified number of questions.
- Job Matching — AI-based comparison of resume and job description.
- Job Application Tracking (Applied Jobs).
- Dashboard summarizing resumes, applications, and interview preparation activity.
- A live, publicly reachable deployment of the frontend at https://carrier-ai-frontend.onrender.com.

11.2 AI Functionality — Discussion

The AI functionality in AI Resume Analyzer is used for four distinct tasks: analyzing resume content, generating resume content, generating interview questions, and comparing a resume against a job description. In each case, the underlying mechanism is the same — extracted or user-provided text is combined into a prompt and sent to the LLM via the Groq API, and the response is used to drive the corresponding feature. Centralizing this through a single AI service means the quality of every AI feature depends heavily on prompt design (see Chapter 7) rather than on separate, feature-specific AI logic.

11.3 Observations

- Combining resume and job-description context for interview questions is what differentiates this system's interview preparation from generic, non-personalized question banks.
- Because there is no confirmed authentication system, resume and application data as currently described appears to be identified by record identifiers rather than tied to individual user accounts — this is discussed further as a limitation and future enhancement.
- The reliance on structured JSON responses from the LLM means the reliability of features such as exact interview-question counts depends on how strictly the LLM follows the requested format, which is a known characteristic of LLM-based systems (see Chapter 12).
- Hosting the frontend and backend as separate services on Render is consistent with common practice for small-to-medium web applications and keeps the two layers independently deployable.

11.4 Alignment With Objectives

Objective (Chapter 1.3)	Outcome
Resume upload with automatic extraction	Implemented — PDF and DOCX upload with text extraction
AI-based analysis and feedback	Implemented — via the AI service and Groq API
AI-assisted and manual resume builders with templates/theme/font	Implemented — both builder modes described and available
AI-generated interview questions from resume + job description	Implemented — question count is user-specified; exact-count reliability is a known limitation
Resume-to-job-description match assessment	Implemented — qualitative match analysis; scoring methodology not confirmed
Job application tracking	Implemented — Applied Jobs module
Summary dashboard	Implemented — consolidated activity overview
Live, publicly accessible deployment	Implemented — frontend hosted at carrier-ai-frontend.onrender.com

Chapter 12 — Challenges and Limitations

12.1 Challenges Faced During Development

- PDF/DOCX text extraction: resumes come in widely varying layouts (columns, tables, graphics-heavy designs), which makes reliable text extraction harder than parsing plain text documents.
- Unstructured resume data: extracted resume text has no guaranteed structure (no consistent section headers or ordering), which the AI service has to work with as free text.
- LLM output consistency: LLM responses can vary between calls even for similar prompts, which affects features that expect a specific format or count, such as interview question count.
- Prompt design: crafting prompts that reliably produce the intended structure, such as valid JSON and exact question counts, required iteration.
- Structured JSON output: ensuring the LLM's response can always be parsed as valid JSON, and handling cases where it cannot.
- File handling: safely accepting, validating, and storing uploaded files of different formats.
- Resume-template rendering: applying AI-generated or manually entered content consistently across multiple templates.
- Maintaining frontend state: keeping the resume preview, selected template/theme/font, and generated content in sync across the builder interfaces.
- Connecting frontend and backend across two independently deployed services: coordinating request/response formats and CORS configuration between the hosted frontend and the FastAPI routes.
- Database persistence: ensuring resume, interview, and job-tracking records are correctly saved and retrievable via SQLAlchemy.
- API errors and validation: handling failures from the Groq API and validating incoming requests consistently across routes.
- Cold-start latency: free or low-tier hosting plans on platforms such as Render can spin services down during inactivity, introducing a noticeable delay on the first request after idle periods.

12.2 Known Issues and Lessons Learned

1. Resume identifiers being difficult for users to identify when filenames or IDs are not human-readable — Lesson: expose a human-readable resume title/label in the UI alongside the internal identifier, rather than showing the raw ID to users.
2. Resume ID fields in Interview Preparation and Job Matching needing better integration with uploaded resumes — Lesson: let users pick from a list of their previously uploaded or built resumes rather than requiring them to know or re-enter an ID.
3. UI state resetting after AI generation — Lesson: preserve builder selections (template, theme, font, instructions) in frontend state across the generation request/response cycle.

4. AI returning fewer interview questions than requested — Lesson: validate the returned count against the requested count server-side, and retry or explicitly prompt the LLM again if the count does not match.
5. Template preview needing to accurately show the selected template — Lesson: ensure the preview rendering path and the final saved or printed rendering path use the same template-rendering logic, so what the user previews is what they get.
6. Template preview and generated resume needing consistent rendering — Lesson: treat resume data and template rendering as separate concerns so any content — AI-generated or manual — renders identically in preview and output.
7. Need for frontend/backend state synchronization — Lesson: rely on the backend as the source of truth for saved resume state, refetching after save rather than assuming frontend state matches backend state indefinitely.
8. Cross-origin requests failing intermittently between the two hosted services — Lesson: explicitly configure allowed origins on the backend rather than relying on permissive defaults during development that may not carry over to production.

12.3 Limitations

- AI output — analysis, generated resume content, interview questions, match analysis — may not always be accurate, complete, or ideally phrased; it reflects the LLM's interpretation of the input, not a guaranteed-correct assessment.
- Resume parsing may fail or produce incomplete text for resumes with unusual layouts, heavy use of graphics, or non-standard formatting.
- LLM responses can vary between runs, even for very similar prompts, which can affect consistency of generated content.
- The depth of ATS-related analysis depends entirely on what is actually implemented in the ATS service module; this report does not assume a specific ATS scoring methodology beyond the module's presence.
- Job matching quality depends heavily on the quality and specificity of both the resume text and the job description provided by the user.
- SQLite, while suitable for development and small-scale use, is not designed for high-concurrency, large-scale production deployments.
- The system depends on the availability and responsiveness of the Groq API; if the API is unreachable or rate-limited, AI-dependent features will not function.
- Free or low-tier hosting on Render may introduce cold-start delays and resource limits not present in a paid production tier.

Chapter 13 — Project Management

13.1 Development Approach

The project was developed incrementally, starting with the core backend (file upload, parsing, and database persistence) before layering in AI-dependent features (analysis, generation, interview questions, matching) and finally the deployment of both frontend and backend as independently hosted services. This mirrors a typical small-team or individual project workflow where foundational, non-AI infrastructure is proven first, since every AI-dependent feature is built on top of it.

13.2 Indicative Project Phases

Phase	Focus	Key Deliverables
Phase 1 — Planning	Requirements gathering, technology selection, architecture design	Requirements list, chosen stack (Chapter 4), architecture sketch (Chapter 5)
Phase 2 — Core Backend	FastAPI project setup, database models, file upload and parsing	Working upload endpoint, SQLite schema, PDF/DOCX text extraction
Phase 3 — AI Integration	Groq API integration, prompt design for analysis/generation/interview/matching	ai_service.py, prompt templates, structured JSON responses
Phase 4 — Resume Builders	AI and manual resume builder UI, template system, live preview	Multiple templates, theme/font selection, save/print/download
Phase 5 — Interview & Matching	Interview question generation, job matching comparison logic	Interview prep screen, job matching screen
Phase 6 — Job Tracking & Dashboard	Applied Jobs module, summary dashboard	Job tracking CRUD, dashboard aggregation views
Phase 7 — Testing & Hardening	Manual test-case verification, error handling improvements	Test case results, improved validation/error handling
Phase 8 — Deployment	Hosting frontend and backend on Render, environment configuration	Live application at carrier-ai-frontend.onrender.com

13.3 Roles and Responsibilities

As an individual or small-team academic project, the same contributor(s) are typically responsible across backend development, AI/prompt engineering, frontend development, and deployment. Where the project was built by a team, this section should be expanded to list each member's name and the module(s) they were primarily responsible for.

13.4 Tools Used in Development

- Version control: a Git-based repository is assumed for source management (exact hosting — GitHub, GitLab, or similar — not specified).

- API testing: FastAPI's automatically generated interactive documentation (Swagger UI / ReDoc) for manually exercising backend endpoints during development.
- Hosting/CI: Render for deployment of both frontend and backend services.
- Local development: Uvicorn's auto-reload mode for the backend, and a local frontend development server for the client.

Chapter 14 — Cost Analysis

14.1 Purpose

This chapter provides an indicative view of the recurring costs associated with running AI Resume Analyzer, to inform decisions about scaling the project beyond an academic or portfolio deployment. Exact figures depend on the specific pricing tiers chosen and on Groq API usage volume, which were not confirmed for this report.

14.2 Cost Components

Cost Component	Description	Notes
Hosting — frontend	Render service hosting the static/SPA frontend build	Free tier available; paid tier removes cold-start delay
Hosting — backend	Render service hosting the FastAPI/Uvicorn process	Free tier available; paid tier provides more consistent uptime and resources
LLM API usage (Groq)	Per-request or token-based cost for analysis, generation, interview, and matching calls	Scales directly with user activity; exact pricing tier not specified
Database storage	SQLite file storage on the backend host	Minimal cost at current scale; a factor if migrating to a managed database
Domain name (optional)	A custom domain instead of the default onrender.com subdomain	Optional; not confirmed as currently in use

14.3 Cost Scaling Considerations

- AI API cost is the most usage-sensitive line item, since every analysis, generation, interview, and matching action triggers an LLM call; rate limiting (Chapter 10.2) also helps control this cost as usage grows.
- Free hosting tiers are appropriate for demonstration and academic evaluation but introduce cold-start latency (Chapter 12.1) that would need to be addressed before a production-facing launch.
- Migrating from SQLite to a managed database (Chapter 15) would introduce a new recurring cost but would be necessary before supporting meaningful concurrent, multi-user production traffic.

Chapter 15 — Future Enhancements

The items below are proposed future improvements. None of them are confirmed as implemented in the current system; they are listed here as directions for future work.

- Authentication and user accounts, so resumes, applications, and interview sessions are tied to individual users.
- Better resume management, including resume versioning and history.
- Improved, more transparent ATS scoring.
- Additional resume templates.
- A drag-and-drop resume editor.
- More accurate and robust resume parsing for varied layouts.
- Semantic (embedding/vector-based) resume-to-job-description matching, if greater matching accuracy is needed than direct LLM comparison provides.
- Job portal integration, to pull job descriptions directly rather than requiring manual entry.
- Automated job recommendations based on a user's resume and history.
- Interview simulation, such as timed or conversational practice.
- Voice-based interview practice.
- A more detailed analytics dashboard.
- Migration to a cloud-hosted, managed database for multi-user, production-scale use.
- A formal, hardened production deployment with monitoring and alerting.
- Improved PDF generation quality for downloaded resumes.
- Resume history and version management across edits.
- A dedicated custom domain and CDN in front of the hosted frontend to reduce latency.
- Automated CI/CD pipelines for the frontend and backend to reduce manual deployment steps on Render.

Chapter 16 — Deployment

This chapter discusses how AI Resume Analyzer is deployed and what would be required to move from the current hosted state toward a more production-hardened deployment.

16.1 Current Deployment

The frontend is deployed and publicly reachable at https://carrier-ai-frontend.onrender.com, hosted on Render. The naming convention of the frontend service is consistent with a companion backend service deployed separately, matching the decoupled architecture described in Chapter 5. The frontend communicates with the backend over HTTPS/JSON to perform all upload, analysis, generation, interview, and matching actions.

16.2 Local Development

In local development, the FastAPI backend is run using Uvicorn, reading configuration — including the Groq API key — from a local .env file, and using a local SQLite file for storage. The frontend is run separately using its own development server and configured to call the local backend's API base URL rather than the hosted one.

16.3 Backend Server

For a hosted deployment, the FastAPI application runs behind an ASGI server (Uvicorn), which Render manages as a long-running web service process. A reverse proxy or load balancer in front of this process, such as Nginx, would be a reasonable addition if traffic grows beyond what a single instance can serve.

16.4 Environment Variables

In any deployment, the Groq API key and other configuration values should continue to be supplied via environment variables — or a secrets manager in a more advanced cloud environment — rather than committed to source control. On Render, this corresponds to the service's environment variable configuration panel rather than a checked-in .env file.

16.5 Database

SQLite is appropriate for local development, demonstration, and small-scale use. For a genuine multi-user production deployment, migrating to a server-based database such as PostgreSQL or MySQL would be a reasonable future step, since SQLite's file-based, single-writer design does not scale well to concurrent multi-user production traffic, and its data would not automatically persist across certain types of redeploys on ephemeral hosting filesystems.

16.6 Frontend Hosting

The frontend is hosted as a separate Render service at the carrier-ai-frontend subdomain. Whether it is served as a static build or through a small server process was not confirmed from the deployed application; this should be filled in directly from the frontend project's deployment configuration.

16.7 Production Considerations

- Migrating from SQLite to a production-grade database for concurrent multi-user access.
- Adding authentication before exposing the system publicly, since job and resume data is personal.
- Adding rate limiting and monitoring around Groq API usage and cost.
- Implementing the error-handling and input-validation improvements listed in Chapter 9.
- Running both services behind HTTPS with correctly scoped CORS configuration, as already required for the frontend and backend to communicate securely.
- Upgrading from any free hosting tier to a paid tier to eliminate cold-start latency for end users.

Note: No claim beyond the confirmed frontend URL is made about the specific production configuration of the backend service; this chapter describes the deployment model and considerations rather than a fully verified production setup.

Chapter 17 — Learning Outcomes

Building AI Resume Analyzer required applying a broad range of software development skills in an integrated, end-to-end way. Key learning outcomes include:

- Practical Python programming applied to a real, multi-feature backend application.
- Building and structuring a REST API using FastAPI, including route design and request/response handling.
- Understanding and applying core REST API concepts: endpoints, HTTP methods, JSON request/response bodies, and status codes.
- Designing a layered backend architecture that separates routes, services, schemas, and the database layer.
- Using SQLAlchemy as an ORM, and understanding how it maps Python objects to relational database tables.
- Working with SQLite as a lightweight relational database suitable for development-scale projects.
- Considerations in relational database design, such as identifying entities and relationships (see Chapter 5.5 and Appendix A).
- File handling in a backend context: accepting, validating, and storing uploaded files.
- Extracting text from PDF and DOCX documents programmatically.
- Integrating a third-party LLM API (Groq) into an application, including authentication via API keys and environment variables.
- Prompt engineering: designing system and user prompts, and using context injection to make LLM output relevant to a specific resume and job description.
- Requesting and working with structured JSON output from an LLM, and understanding why this matters for backend integration.
- Coordinating frontend and backend integration across two independently deployed services, including CORS configuration and API base URL management.
- Basic UI/UX considerations for a multi-screen application: dashboard, builders, analysis, interview preparation, matching, and tracking.
- State management within a frontend application, particularly for live-updating previews.
- Designing error handling for a range of failure scenarios, including third-party API failures.
- Planning a testing strategy and writing meaningful test cases for a feature-rich application.
- Deploying a full-stack application to a cloud PaaS (Render), including environment configuration for a live, publicly reachable system.
- General debugging skills across a full-stack, AI-integrated, and deployed application.
- Broader software architecture thinking: separating concerns across layers so the system remains maintainable as features are added.

- Practical experience in AI application development, specifically applying LLMs to real, non-trivial tasks rather than toy examples.

Chapter 18 — Conclusion

AI Resume Analyzer brings together resume upload and parsing, AI-based resume analysis, AI-assisted and manual resume building with template, theme, and font customization, AI-generated interview preparation driven by resume and job-description context, AI-based job matching, and job application tracking, within a single FastAPI-based backend and SQLite database, using the Groq API as the LLM access layer, fronted by a browser-based client hosted at https://carrier-ai-frontend.onrender.com.

The project demonstrates how an LLM can be applied to a genuinely useful, everyday problem — job search preparation — by using it not as a generic chatbot, but as a component that reads specific, user-provided context, such as a resume, a job description, or builder instructions, and produces structured, task-specific output. This required careful attention to prompt design and structured JSON output, in addition to standard backend engineering practices such as API design, database modeling, file handling, and deployment.

From a technical-skills perspective, the project involved practical experience across backend API development (FastAPI/Uvicorn), database design and access (SQLAlchemy/SQLite), document processing (PDF/DOCX parsing), third-party AI API integration (Groq), frontend and backend deployment on a cloud PaaS, and the software architecture discipline of separating routes, services, and data access layers.

As documented in this report, several aspects of the system — the exact database schema, the exact API endpoints, the specific frontend technology used, and the confirmed security and testing status — could not be verified without direct inspection of the project's source code, and are marked accordingly rather than assumed. The features described, and the fact that the application is live and publicly reachable, represent a coherent and practically useful application of AI to resume preparation and job search readiness, with clear directions for future enhancement (Chapter 15), particularly around authentication, more advanced matching techniques, and production-readiness.

References

The following are general references relevant to the technologies used in this project. Specific citations, such as exact library versions or official documentation links used during development, were not provided and are not fabricated here.

- FastAPI official documentation — https://fastapi.tiangolo.com/
- Uvicorn official documentation — https://www.uvicorn.org/
- SQLAlchemy official documentation — https://www.sqlalchemy.org/
- SQLite official documentation — https://www.sqlite.org/
- Groq API / Groq Python client documentation — https://groq.com/
- python-dotenv documentation — https://pypi.org/project/python-dotenv/
- Python official documentation — https://docs.python.org/3/
- Render platform documentation — https://render.com/docs
- AI Resume Analyzer (CareerAI) — live application — https://carrier-ai-frontend.onrender.com

Note: Add any additional references actually used during development — tutorials, community threads, or specific parsing/rendering libraries once confirmed — before final submission.

Appendix A — Database Design (Detailed)

The project uses SQLite as its database engine, accessed through the SQLAlchemy ORM, with model classes defining each table and a separate module configuring the database connection and session.

A.1 What SQLite Provides Here

SQLite stores the entire database as a single file. This is well suited to this project because it needs no separate database server process, which simplifies local development and small-scale deployment, while still supporting standard relational operations — tables, primary keys, foreign keys, and queries — through SQLAlchemy.

A.2 What SQLAlchemy Provides Here

SQLAlchemy allows each database table to be defined as a Python class, referred to as a model, with each row represented as an instance of that class. This means the rest of the backend code can create, read, update, and query records using normal Python objects and method calls, rather than writing raw SQL statements throughout the codebase. This also makes validation, relationships between tables, and future schema changes easier to manage centrally.

A.3 Exact Tables, Columns, and Relationships

The exact table names, column names, data types, primary keys, and foreign key relationships are defined in the backend's model definitions, which were not provided for direct inspection when this report was generated. Rather than inventing table or column names, this report lists the entities that the described features imply must be persisted in some form, and marks the schema itself as not specified.

Entity (implied by features)	Likely Purpose	Exact Schema
Resume record	Associates an uploaded or built resume with a resume identifier, its extracted/generated content, and metadata such as template, theme, and font.	Not specified — confirm from model definitions
Interview preparation record	Associates a resume and job description with generated interview questions and the requested question count.	Not specified — confirm from model definitions
Job match record	Associates a resume and job description with a match analysis result.	Not specified — confirm from model definitions
Job application (tracking) record	Stores tracked job applications, such as company, title, status, and date.	Not specified — confirm from model definitions
User record (future)	Would associate the above entities with an individual authenticated user, once authentication is implemented.	Not implemented — see Chapter 15

Note: This appendix should be replaced with the actual table definitions — column names, types, keys, and relationships — extracted directly from the backend's model definitions once the source code is available.

Appendix B — API Documentation (Template)

The backend exposes its functionality through FastAPI routes, organized by feature. The exact endpoint paths, HTTP methods, request bodies, and response shapes were not provided for inspection, so this appendix provides a template — one row per described feature — to be completed directly from the actual route definitions rather than invented endpoint names.

Feature (route module)	Likely Method	Purpose
Resume upload	POST	Upload a resume file (PDF/DOCX) and store it
Resume analysis	POST / GET	Request AI-based analysis of a stored resume
Resume improvement / AI generation	POST	Generate or improve resume content using AI
Interview preparation	POST	Generate interview questions from a resume, job description, and question count
Job matching	POST	Compare a resume to a job description
Job tracking	POST / GET / PUT / DELETE	Create, list, update, and remove tracked job applications
Dashboard summary	GET	Return aggregate counts and recent activity for the dashboard

For each actual endpoint, the final report should document the exact endpoint path, HTTP method, purpose, request parameters or body schema, response schema, and possible error responses, inspected directly from the route and schema definitions. The following template can be used per endpoint:

```
Endpoint:        POST /api/<feature>/<action>
Description:     <one-line purpose>
Request body:    { field1: type, field2: type, ... }
Response body:   { field1: type, field2: type, ... }
Success status:  200 / 201
Error statuses:  400 (validation), 404 (not found), 500 (server/AI failure)
```

Appendix C — Technical Skills Demonstrated

Skill	How It Was Used in the Project
Python	Implemented the entire backend application logic.
FastAPI	Defined and exposed the application's API routes.
REST API design	Structured feature access (upload, analyze, improve, interview, jobs) as HTTP endpoints.
SQLAlchemy	Mapped application data (resumes, interviews, job records) to database tables as Python models.
SQLite	Provided the file-based relational database for the application.
Groq API	Provided programmatic access to an LLM for analysis, generation, interview questions, and matching.
LLM usage	Applied an LLM to read resume and job-description text and produce structured, task-specific output.
Prompt engineering	Designed system/user prompts with resume, job description, and instruction context injection.
PDF parsing	Extracted text content from uploaded PDF resumes.
DOCX parsing	Extracted text content from uploaded DOCX resumes.
JSON	Used as the structured format for AI responses and API request/response bodies.
Environment variables (.env)	Kept the Groq API key and configuration out of source code.
Database design	Structured resume, interview, and job-tracking data as related entities (see Appendix A).
Frontend/backend integration	Connected a separately hosted frontend to backend endpoints for all features.
File handling	Managed upload, storage, and retrieval of resume files.
Cloud deployment (Render)	Deployed both frontend and backend as independently hosted, publicly reachable services.
AI application development	Built multiple LLM-driven features within a single coherent application.
Debugging	Diagnosed and resolved issues such as those listed in Chapter 12, including mismatched interview-question counts.
Testing (strategy)	Defined test cases across upload, analysis, building, interview prep, matching, and database operations (Chapter 9).

Appendix D — Glossary of Terms

Term	Meaning
ATS	Applicant Tracking System — software used by employers/recruiters to collect, filter, and rank incoming resumes.
LLM	Large Language Model — an AI model trained on large amounts of text, capable of reading a prompt and generating relevant text output.
API	Application Programming Interface — a defined way for one piece of software to request functionality or data from another.
ORM	Object-Relational Mapper — a library that maps database tables to programming-language objects (SQLAlchemy, in this project).
ASGI	Asynchronous Server Gateway Interface — the interface standard that lets an async Python web framework run under a compatible server (Uvicorn, in this project).
Prompt engineering	The practice of designing the text sent to an LLM so its response is accurate, relevant, and in the expected format.
Context injection	Inserting user- or system-specific data (such as resume text or a job description) into a prompt so the LLM's output is grounded in that data.
SPA	Single-Page Application — a web application that loads a single HTML page and updates content dynamically via JavaScript rather than full page reloads.
PaaS	Platform as a Service — a cloud hosting model (Render, in this project) that manages the underlying servers so developers deploy applications directly.
CORS	Cross-Origin Resource Sharing — a browser security mechanism that controls whether a frontend on one origin may call a backend on another origin.
Cold start	The delay experienced when a hosted service that has been idle needs to restart before it can serve a new request.

Appendix E — User Manual (Quick Start Guide)

E.1 Accessing the Application

1. Open a modern web browser (Chrome, Edge, Firefox, or Safari).
2. Navigate to https://carrier-ai-frontend.onrender.com.
3. Wait for the application to load; on a free hosting tier, the first load after a period of inactivity may take longer than usual (see Chapter 12.1).

E.2 Analyzing an Existing Resume

1. From the main navigation, open the Resume Analysis screen.
2. Upload an existing resume in PDF or DOCX format.
3. Submit the file and wait for the system to extract the text and run the AI-based analysis.
4. Review the structured feedback returned and use it to revise the resume as needed.

E.3 Building a Resume With AI Assistance

1. Open the AI Resume Builder screen.
2. Choose a template (for example, Classic Sidebar, Dark Banner, or Minimal), a theme color, and a font.
3. Enter a resume title and provide instructions describing the desired content — career background, target role, and any specific points to include.
4. Submit the request and review the AI-generated content in the live preview.
5. Save the resume, then print or download it as needed.

E.4 Building a Resume Manually

1. Open the Resume Builder (Manual) screen.
2. Choose a template, theme color, and font.
3. Enter resume content directly into the provided fields.
4. Review the live preview, save the resume, then print or download it as needed.

E.5 Preparing for an Interview

1. Open the Interview Preparation screen.
2. Select or reference a previously uploaded or built resume.
3. Paste the target job description into the provided field.
4. Specify the number of interview questions to generate.
5. Submit the request and review the generated questions along with their suggested answer guidance.

E.6 Matching a Resume to a Job Description

1. Open the Job Matching screen.
2. Select or reference a resume and paste the target job description.
3. Submit the request and review the AI-generated match analysis, including areas of alignment and gaps.

E.7 Tracking Job Applications

1. Open the Applied Jobs screen.
2. Add a new job application record with details such as company, role, status, and date.
3. Update the status of existing records as applications progress.

E.8 Using the Dashboard

Return to the Dashboard at any time to see a summary of resumes uploaded or built, jobs applied to, interviews prepared, and quick links back into each feature area.

Appendix F — Verification Checklist Before Final Submission

Because several implementation details could not be confirmed from source-code inspection at the time this report was generated, the following checklist should be completed against the actual codebase and live deployment before this report is treated as final.

- Confirm the exact frontend framework/library and record it in Chapter 4.9.
- Confirm the exact PDF and DOCX parsing libraries used and record them in Chapter 4.7 and Chapter 6.3.
- Confirm the specific Groq LLM model name/version used and record it in Chapter 4.6.
- Extract and document the actual SQLAlchemy model definitions, replacing the placeholders in Appendix A.
- Extract and document the actual FastAPI route paths, methods, and schemas, replacing the placeholders in Appendix B.
- Insert real screenshots of each screen listed in Chapter 8.2.
- Run the test cases in Chapter 9 against the live and/or local deployment and record actual pass/fail results.
- Confirm which security items in Chapter 10.2 are actually implemented versus outstanding.
- Confirm the backend service's hosting URL and add it alongside the frontend URL throughout this report.
- Fill in the degree name, student name(s), institution, and academic year on the title page.